

DataFlix — Dataset & Feature Analysis

1. Dataset-by-Dataset Analysis

Netflix Kaggle Dataset (archive/)

File	Shape	Purpose
Netflix_Dataset_Rating.csv	17,337,458 × 3	User-movie interaction matrix
Netflix_Dataset_Movie.csv	17,770 × 3	Movie identity/metadata

- `Netflix_Dataset_Rating.csv`: Columns = `User_ID`, `Rating`, `Movie_ID`. Ratings are integers 1–5, no timestamps, zero nulls. Critical weakness: No date column — temporal train/test split is impossible.
- `Netflix_Dataset_Movie.csv`: Columns = `Movie_ID`, `Year`, `Name`. Year range: 1915–2005, zero nulls. Very lightweight metadata — only title and year.

 **Key finding:** The Netflix rating file covers only 1,350 unique movies out of 17,770 in the movie file. The movie file is almost entirely uninteracted (87% of movies have zero ratings in this Kaggle version).

MovieLens 25M (ml-25m/)

File	Shape	Purpose
<code>ratings.csv</code>	25,000,095 × 4	Dense interaction matrix
<code>movies.csv</code>	62,423 × 3	Movie identity + genres
<code>links.csv</code>	62,423 × 3	IMDb/TMDb ID bridge
<code>tags.csv</code>	1,093,360 × 4	User-written free-text tags
<code>genome-scores.csv</code>	15,584,448 × 3	Tag relevance (1128 tags × 13,816 movies)
<code>genome-tags.csv</code>	1,128 × 2	Tag vocabulary

- `ratings.csv`: `userId`, `movieId`, `rating`, `timestamp`. Ratings 0.5–5.0 (half-step), zero nulls. Has timestamp → temporal split is possible.
- `movies.csv`: `movieId`, `title`, `genres`. 19 clean genres as pipe-separated strings.

- `links.csv`: movied, imdbld, tmdbld. This is the bridge to TMDb for fetching plot synopses.
- `genome-scores.csv`: 15.5M rows — a $13,816 \times 1,128$ relevance matrix per movie. Very powerful signal but extremely large — costs 500MB+ in RAM.

TMDb API / Local Data

- Provides: plot overview, release date, original language, runtime, `vote_average`, `vote_count`, popularity, `spoken_languages`, tagline.
- Accessed using `links.csv` → `tmdbld` as the bridge key.
- This is the source of text for SBERT embeddings.

2. Recommended Dataset Usage

Dataset	Use?	Role
Netflix Rating CSV	✓ YES	Validation + Testing only (no user overlap possible with ML training)
Netflix Movie CSV	✓ YES	Movie title/year lookup for alignment
MovieLens <code>ratings.csv</code>	✓ YES	Primary training interaction matrix
MovieLens <code>movies.csv</code>	✓ YES	Genre feature table
MovieLens <code>links.csv</code>	✓ YES	Bridge to TMDb API
MovieLens <code>tags.csv</code>	⚠️ OPTIONAL	Can supplement SBERT if plot is missing
MovieLens <code>genome-scores.csv</code>	✗ NO	Too large (15.5M rows), 13,816 movies only, overlaps with SBERT
TMDb Data	✓ YES	Plot synopses for SBERT embeddings
IMDb / OMDb	✗ NO	Redundant — TMDb already provides runtime, language, genres

3. Merge Strategy

1. Netflix Movie CSV $\longleftrightarrow$ MovieLens `movies.csv` (Key: Cleaned title + Year, fuzzy TF-IDF)
2. MovieLens `movies.csv` $\longleftrightarrow$ MovieLens `links.csv` (Key: `movied`)
3. `movied` → `tmdbld` → TMDb overview
4. All merged on unified `movie_idx` (contiguous int).

 **Warning:** Never merge Netflix Movie_ID directly with MovieLens movielid — they are completely different internal ID spaces. Always go through the title+year fuzzy map.

4. Column Classification Table

Dataset	Column	Classification	Use / Remove	Reason	Effect on Accuracy	Effect on Speed
Netflix Rating	User_ID	Essential	USE	Primary identifier for val/test user lookup	High	Neutral
Netflix Rating	Rating	Essential	USE	Target variable for RMSE evaluation	High	Neutral
Netflix Rating	Movie_ID	Essential	USE (mapped)	Links to ML movie space	High	Neutral
Netflix Movie	Movie_ID	Essential	USE	Key for fuzzy alignment	High	Neutral
Netflix Movie	Name	Essential	USE	Title matching in alignment	High	Neutral
Netflix Movie	Year	Useful Optional	USE	Reduces false-positives	Medium	Neutral
ML Ratings	userId	Essential	USE	Defines training user profiles	High	Neutral
ML Ratings	movielid	Essential	USE	Defines item space	High	Neutral
ML Ratings	rating	Essential	USE	Target for MSE / Path A	High	Neutral
ML Ratings	timestamp	Essential	USE	Required for temporal train/val split	High	Neutral
ML Movies	movielid	Essential	USE	Join key	High	Neutral
ML Movies	title	Useful Optional	USE	Title text used only in fuzzy match	Low	Neutral

Dataset	Column	Classification	Use / Remove	Reason	Effect on Accuracy	Effect on Speed
ML Movies	genres	Essential	USE	Genre embedding table	High	Low cost
ML Links	movied	Essential	USE	Bridge key	High	Neutral
ML Links	imdbld	Unnecessary	REMOVE	TMDb covers everything IMDb provides	Neutral	Neutral
ML Links	tmdbld	Essential	USE	API key to fetch plots	High	Neutral
Genome Scores	movied	Harmful	REMOVE	Only covers 13,816/62,423 movies	Negative	Very slow
Genome Scores	tagld	Harmful	REMOVE	1,128-dim vector dimension explosion	Marginal	Very slow
TMDb	overview	Essential	USE	Primary SBERT input text	Very High	Neutral
TMDb	popularity	Essential	USE	Scalar feature for hybrid model	Medium	Neutral
TMDb	vote_average	Harmful	REMOVE	Aggregate rating signal — leaks test labels	Leakage	Neutral

5. Leakage and Harmful Columns

Column	Risk	Why
vote_average (TMDb)	 DIRECT LEAKAGE	Aggregate of all user ratings — is essentially a smoothed version of the target we are trying to predict
vote_count (TMDb)	 Indirect Leakage	Highly correlated with popularity rank which correlates with rating distribution

Column	Risk	Why
genome-scores relevance	⚠ Noise Risk	Only available for 13,816/62,423 movies. Applying it creates structured missingness that biases the model
Netflix Movie_ID	❌ WRONG KEY	Netflix IDs are internal to Netflix and different from ML IDs

6. Required Preprocessing Steps

1. Temporal split on MovieLens only (80% train / 10% val / 10% test).
2. Random user-level split on Netflix (50% val / 50% test).
3. Filter ML sparse interactions: Remove users with < 5 ratings; movies with < 10 ratings.
4. Fuzzy title alignment Netflix → MovieLens.
5. Drop rows where Netflix movie_id is not in the alignment map.
6. Mean-center ratings per user for MF Path A.
7. BPR binarisation: centered rating > 0 → positive interaction.
8. Genre one-hot / embedding.
9. SBERT on TMDb overview field.
10. Log-normalise popularity scalar.
11. Pop-proportional negative sampling for BPR.

7. Recommended Features Per Model Component

Matrix Factorization (Path A) - user_idx (contiguous int) - movie_idx (contiguous int) - rating_centered (float32 target)

BPR Ranking (Path B) - user_idx - positive_movie_idx - negative_movie_idx (sampled popularity-proportional)

SBERT Embeddings - Primary: TMDb overview (plot text) - Fallback if missing: ML movie title + genres

Metadata Features (per movie) - Genre embedding: ML movies.csv genres - Popularity scalar: Log(ML rating count + 1) - Release year: Netflix Movie Year or TMDb release_date

8. Final Strict Decision

-  **Use:** Netflix_Dataset_Rating.csv (validation/testing only), MovieLens ratings.csv (training), MovieLens movies.csv, MovieLens links.csv, TMDb: overview, popularity, release_date(year).
-  **Do Not Use:** genome-scores.csv, tags.csv, genome-tags.csv, TMDb vote_average, TMDb vote_count, IMDb.
-  **Merge By:** Netflix Movie_ID → MovieLens movieId via TF-IDF fuzzy title+year match. MovieLens movieId → TMDb via links.csv tmdbId exact join.
- **Required Columns:** user_idx, movie_idx, rating, timestamp(ML only), genres(ML), tmdbId(links), tmdb.overview.
- **Remove Columns:** vote_average, vote_count, tagline, spoken_languages, imdbId, genome-scores.*
- **Final Verdict:** The data is highly sufficient for the proposed DataFlix hybrid architecture!